Part 1: Theoretical Questions

1. (a) Explanation of the following paradigms:
   a. Imperative programming paradigm is based on the idea that the control flow is an explicit sequence of commands. For example Java language is imperative.
   ii. Procedural programming is as same as Imperative, but organized around hierarchies of nested procedure calls. For example Java language is procedural.
   iii. Functional programming - Computation proceeds by (nested) function calls that avoid any global state mutation and through the definition of function composition. The program is an expression, or a series of expressions, and not a sequence of commands. Running the program is calculating the expression(s), finding its value, and not executing the commands. Functions are also expressions. A function call is the evaluation of an expression. Since a function is also an expression, a function can be passed as a parameter to another procedure, as well as returned as a value from a function procedure. For example Scheme language is functional.

   (b) The procedural paradigm improve over the imperative paradigm by adding layers of abstraction in the form of procedures. Procedures interact through well defined contracts and can encapsulate local variables.

   (c) The functional paradigm improves over the procedural paradigm by discouraging the use of shared state and mutation, which makes testing, formal verification, and concurrency easier.

2. Here is the functional approach for the given function getDiscountedProductAveragePrice:

```
const getDiscountedProductAveragePrice = (inventory: Product[]): number =>
    inventory
      .filter(product => product.discounted)
      .reduce((acc, product, _, arr) => acc + product.price / arr.length, 0)
```

3. The most specific types for the following expressions:
   a. (x, y) => x.some(y) will be:
      <T>(x: T[], y: (z: T) => boolean) => boolean
   b. x => x.reduce((acc, cur) => acc + cur, 0) will be:
      (x: number[]) => number
   c. (x, y) => x ? y[0] : y[1] will be:
      <T>(x: boolean, y: T[]) => T
   d. (f,g) => x => f(g(x+1)) will be:
      <T1, T2>(f: (b: T1) => T2, g: (a: number) => T1) => (x: number) => T2